someone to write down a value, you increase the batch number: “Please write down 41, as part of batch 424.”

When you ask for a value, you now have a choice: you can either just ask for a value (which is a relaxed load), in which case the man just gives you the number, or you can ask for a value and information about whether it’s the last in a batch (which models a load-acquire). If you ask for the batch information, and the value wasn’t the last in a batch, the man will tell you something like, “The number is 987, which is just a ‘normal’ value,” whereas if it *was* the last in a batch, he’ll tell you something like “The number is 987, which is the last number in batch 956 from Anne.” Now, here’s where the acquire-release semantics kick in: if you tell the man all the batches you know about when you ask for a value, he’ll look down his list for the last value from any of the batches you know about and either give you that number or one further down the list.

How does this model acquire-release semantics? Let’s look at our example and see. First off, thread `a` is running `write_x_then_y` and says to the man in cubicle `x`, “Please write `true` as part of batch 1 from thread `a`,” which he duly writes down. Thread `a` then says to the man in cubicle `y`, “Please write `true` as the last write of batch 1 from thread `a`,” which he duly writes down. In the meantime, thread `b` is running `read_y_then_x`. Thread `b` keeps asking the man in box `y` for a value with batch information until he says “`true`.” He may have to ask many times, but eventually the man will say “`true`.” The man in box `y` doesn’t *just* say “`true`” though; he also says, “This is the last write in batch 1 from thread `a`.”

Now, thread `b` goes on to ask the man in box `x` for a value, but this time he says, “Please can I have a value, and by the way I know about batch 1 from thread `a`.” So now, the man from cubicle `x` has to look down his list for the last mention of batch 1 from thread `a`. The only mention he has is the value `true`, which is also the last value on his list, so he *must* read out that value; otherwise, he’s breaking the rules of the game.

If you look at the definition of *inter-thread happens-before* back in section 5.3.2, one of the important properties is that it’s transitive: *if A inter-thread happens-before B* and *B inter-thread happens-before C*, then *A inter-thread happens-before C*. This means that acquire-release ordering can be used to synchronize data across several threads, even when the “intermediate” threads haven’t actually touched the data.

Transitive Synchronization With Acquire-Release Ordering

In order to think about transitive ordering, you need at least three threads. The first thread modifies some shared variables and does a store-release to one of them. A second thread then reads the variable subject to the store-release with a load-acquire and performs a store-release on a second shared variable. Finally, a third thread does a load-acquire on that second shared variable. Provided that the load-acquire operations see the values written by the store-release operations to ensure the synchronizes-with relationships, this third thread can read the values of the other variables stored by the first thread, even if the intermediate thread didn’t touch any of them. This scenario is shown in the next listing.

Listing 5.9. Transitive synchronization using acquire and release ordering